

Microsoft Agent Framework Tutorial

Includes MCP, multi-agent patterns, evaluation, monitoring with Microsoft Foundry, examples, use cases, analogies, and Python snippets.

Microsoft Agent Framework Tutorial

This PDF combines the earlier tutorial on Microsoft Agent Framework and the follow-up explanation of async and await in the Python snippets.

Part 1: Tutorial: Building Enterprise Agents with Microsoft Agent Framework, MCP, Multi-Agent Patterns, Evaluation, and Monitoring

1. Mental model: what is Microsoft Agent Framework?

Think of Microsoft Agent Framework as an enterprise-grade operating system for AI agents.

A normal LLM app is like a smart person sitting in a room answering questions. An agent is like that person with tools, memory, instructions, access control, and a task loop. A workflow is like a business process where multiple specialists work together in a defined sequence.

Microsoft Agent Framework has two major capability areas:

Capability	What it means	When to use
Agents	LLM-powered components that can reason, call tools, use MCP servers, and respond	Open-ended tasks, Q&A, planning, tool use
Workflows	Graph-based orchestration across agents/functions with routing, checkpointing, and human-in-the-loop	Multi-step enterprise processes with control and auditability

Microsoft's docs describe Agent Framework as supporting individual agents that use LLMs, tools, MCP servers, and workflows that connect agents/functions for multi-step tasks with type-safe routing, checkpointing, and human review support.

2. Running use case for this tutorial

We will use a single enterprise use case throughout:

Enterprise Knowledge and Action Assistant

An internal assistant helps application support engineers investigate incidents for an Azure-hosted enterprise application. It can search documentation, inspect runbooks, query telemetry, draft incident summaries, suggest remediation, and escalate to humans when needed.

This maps well to RAG and Azure architecture interests:

Requirement	Agent concept
Answer technical questions from internal docs	RAG tool / Azure AI Search tool

Requirement	Agent concept
Query live system health	MCP tool or custom function tool
Analyze logs and traces	Specialist diagnostic agent
Create remediation plan	Planner agent
Validate response quality	Evaluation
Track latency, token usage, failures	Foundry monitoring
Handle risky actions	Human-in-the-loop workflow

3. Basic agent

A basic agent has:

1. Model - GPT model or other supported provider
2. Instructions - system behavior
3. Tools - APIs/functions/MCP servers
4. Runtime loop - decides when to answer or call tools
5. Telemetry/evaluation - measures quality and reliability

Microsoft Agent Framework supports providers including Microsoft Foundry, Azure OpenAI, OpenAI, Anthropic, Ollama, and others.

Example: simple support assistant

```
import asyncio
from agent_framework import Agent
from agent_framework.foundry import FoundryChatClient
from azure.identity.aio import AzureCliCredential

async def main():
    async with AzureCliCredential() as credential:
        client = FoundryChatClient(
            credential=credential,
            # project endpoint/config typically comes from environment or client config
        )

    async with Agent(
        client=client,
        name="SupportAssistant",
        instructions="""
You are an enterprise application support assistant.
Be concise, cite evidence when available, and avoid making changes
unless explicitly approved by a human.
"""
    ) as agent:
        result = await agent.run(
            "A user reports that login is slow. What should I check first?"
        )
        print(result)

if __name__ == "__main__":
    asyncio.run(main())
```

Analogy

A basic agent is like a junior support engineer: it can answer questions, but without tools it relies mostly on model knowledge. For enterprise work, tools are what make it useful.

4. Tools: giving agents controlled capabilities

Tools let agents interact with external systems: search indexes, APIs, databases, ticketing systems, telemetry systems, and internal services. Microsoft describes tools as extensions that allow agents to interact with external systems, execute code, search data, and more.

Example: custom Python tool

Use this when the system is simple and you own the API.

```
from typing import Annotated
from agent_framework import tool

@tool
def get_service_health(
    service_name: Annotated[str, "Name of the application service"]
) -> str:
    """
    Return simplified health status for a service.
    In production, call Azure Monitor, App Insights, Grafana, ServiceNow, etc.
    """
    mock_health = {
        "auth-service": "degraded: p95 latency is 4.8s, error rate 2.1%",
        "payment-service": "healthy",
        "search-service": "warning: CPU above 80%"
    }

    return mock_health.get(service_name, "unknown service")
```

Use it with an agent:

```
async with Agent(
    client=client,
    name="OpsAgent",
    instructions="You diagnose application health issues.",
    tools=[get_service_health],
) as agent:
    result = await agent.run(
        "Check auth-service health and explain what might be wrong."
    )
    print(result)
```

When to use function tools

Use custom tools when:

Use case	Example
You own the API	Internal incident API
You need strict schema	Change request creation
You want clear access control	Only expose approved operations
You need audit logging	Every tool call gets logged

5. MCP: Model Context Protocol

What is MCP?

Model Context Protocol, or MCP, is an open standard for exposing tools and contextual data to LLM applications. Microsoft Agent Framework supports MCP servers so agents can access external tools and services consistently.

Analogy

MCP is like a universal adapter plug for agents.

Without MCP, every tool integration is custom wiring. With MCP, tools expose a common protocol, and agents can discover and call them in a standard way.

MCP in the enterprise support use case

Need	MCP server example
Read Microsoft documentation	Microsoft Learn MCP
Query GitHub repos	GitHub MCP
Inspect tickets	ServiceNow/Jira MCP
Search internal docs	Custom enterprise MCP
Query cloud resources	Azure MCP server
Read SQL metadata	Database MCP

Local or remote MCP?

MCP type	Best for
Local stdio MCP	Developer tools, local scripts, filesystem, local repo analysis
HTTP/SSE MCP	Enterprise-hosted tools, central services, shared APIs
Foundry-hosted MCP	Managed execution, governed access, production control

Microsoft's docs show Agent Framework support for local stdio MCP and HTTP/SSE MCP connections.

Example: connect an agent to an HTTP MCP server

This example uses the Microsoft Learn MCP endpoint pattern from Microsoft docs.

```
import asyncio
from agent_framework import Agent, MCPStreamableHTTPTool
from agent_framework.foundry import FoundryChatClient
from azure.identity.aio import AzureCliCredential

async def main():
    async with AzureCliCredential() as credential:
        client = FoundryChatClient(credential=credential)

        async with (
            MCPStreamableHTTPTool(
                name="Microsoft Learn MCP",
                url="https://learn.microsoft.com/api/mcp",
            ) as learn_mcp,
            Agent(
                client=client,
                name="DocsAgent",
                instructions="""
                You answer Microsoft documentation questions.
                Use the MCP documentation tool when needed.
                """
            ) as agent,
        ):
            result = await agent.run(
                "How do I monitor agents in Microsoft Foundry?",
                tools=learn_mcp,
            )
            print(result)

if __name__ == "__main__":
    asyncio.run(main())
```

Security warning for MCP

MCP servers can receive prompt content, headers, and contextual data. Microsoft recommends reviewing what data is shared with MCP servers, preferring trusted providers, and logging data for audit when using third-party MCP servers.

For enterprise deployments:

Control	Recommendation
Authentication	Use Entra ID/OAuth or managed identity
Authorization	Expose least-privilege tools only
Data boundary	Avoid sending confidential data to untrusted MCP servers
Audit	Log prompt, tool call, result, identity, timestamp
Approval	Require human approval for write/delete/remediation actions

6. Single agent vs workflow

Microsoft's guidance is simple:

Use an agent when...	Use a workflow when...
Task is open-ended	Process has defined steps
Agent needs to reason and choose tools	You need explicit control
One agent can handle the job	Multiple agents/functions must coordinate
Conversation matters	Auditability and repeatability matter

Microsoft explicitly says to use agents for open-ended/conversational tasks and workflows for well-defined multi-step processes. It also advises: if a function can handle the task, use a function instead of an AI agent.

7. Multi-agent patterns

Microsoft Agent Framework supports several built-in multi-agent orchestration patterns: sequential, concurrent, handoff, group chat, and magentic orchestration.

Pattern map

Pattern	Analogy	Best use case	Example in support assistant
Sequential	Assembly line	Steps must happen in order	Retrieve docs -> analyze logs -> draft RCA
Concurrent	Parallel investigation team	Independent tasks can run together	One agent checks logs, another checks docs, another checks tickets
Handoff	Call center routing	Different expert depending on context	Billing issue goes to BillingAgent, login issue to AuthAgent
Group chat	War room	Agents debate/collaborate	Architect, SRE, Security, App SME discuss incident
Magentic	Team manager	Dynamic decomposition and delegation	Manager agent decides which specialist agents to invoke

7.1 Sequential pattern

Use case

A support investigation must follow a controlled path:

1. Classify issue
2. Retrieve relevant runbook
3. Query system health
4. Generate diagnosis

5. Ask human before remediation

Analogy

Sequential orchestration is a factory assembly line. Each station adds something before passing the work forward.

Pseudocode-style Python

Exact orchestration APIs may vary by package version, but the design looks like this:

```
async def incident_workflow(user_issue: str):
    classification = await classifier_agent.run(user_issue)

    runbook = await knowledge_agent.run(
        f"Find runbook for this issue: {classification}"
    )

    health = await ops_agent.run(
        f"Check service health related to: {classification}"
    )

    diagnosis = await diagnosis_agent.run(f"""
    User issue: {user_issue}
    Classification: {classification}
    Runbook: {runbook}
    Health: {health}

    Produce likely root cause and next steps.
    """)

    return diagnosis
```

When to choose it

Use sequential orchestration when the process is auditable and each step depends on previous output.

7.2 Concurrent pattern

Use case

During an incident, you want faster investigation:

```
User says: "Login is slow."
```

Run three agents in parallel:

1. LogsAgent checks application logs
2. MetricsAgent checks latency/error rates
3. DocsAgent retrieves relevant runbook

Analogy

Concurrent orchestration is like sending three engineers to investigate different rooms at the same time.

```
import asyncio

async def concurrent_investigation(issue: str):
    logs_task = logs_agent.run(f"Analyze logs for: {issue}")
    metrics_task = metrics_agent.run(f"Analyze metrics for: {issue}")
    docs_task = docs_agent.run(f"Find runbooks for: {issue}")

    logs, metrics, docs = await asyncio.gather(
        logs_task,
        metrics_task,
        docs_task,
    )

    summary = await incident_manager_agent.run(f"""
Combine the investigation results.

Issue:
{issue}

Logs:
{logs}

Metrics:
{metrics}

Runbooks:
{docs}

Return root cause hypothesis, confidence, and next actions.
""")

    return summary
```

When to choose it

Use concurrent orchestration when subtasks are independent and latency matters.

7.3 Handoff pattern

Use case

The first agent triages the issue and routes to the right expert.

User request	Handoff target
"Login failed after SSO change"	IdentityAgent
"Payments are stuck"	PaymentsAgent
"CPU is high"	InfraAgent
"Can we apply a hotfix?"	ChangeApprovalAgent

Analogy

Handoff orchestration is like a hospital triage desk routing a patient to the right specialist.

```

async def route_issue(issue: str):
    route = await triage_agent.run(f"""
    Classify the issue into one of:
    identity, payments, infrastructure, security, unknown.

    Issue: {issue}
    Return only the category.
    """)

    route = str(route).strip().lower()

    if route == "identity":
        return await identity_agent.run(issue)
    elif route == "payments":
        return await payments_agent.run(issue)
    elif route == "infrastructure":
        return await infra_agent.run(issue)
    elif route == "security":
        return await security_agent.run(issue)
    else:
        return await general_support_agent.run(issue)

```

When to choose it

Use handoff when the next agent depends on classification or context.

7.4 Group chat pattern

Use case

For critical incidents, multiple experts collaborate:

Agent	Role
SREAgent	Reliability and uptime
SecurityAgent	Data/security impact
AppAgent	Application behavior
CommsAgent	Customer/internal communication
IncidentCommanderAgent	Final decision

Analogy

Group chat is a war room.

Use it when you need diverse perspectives and critique before final output.

7.5 Magentic pattern

Use case

The user asks:

"Investigate why checkout failures increased today, determine the likely root cause, and propose a mitigation plan."

A manager agent dynamically decides:

1. Need metrics
2. Need logs
3. Need recent deployments
4. Need known incident history

5. Need remediation plan

Analogy

Magentic orchestration is like a project manager assigning work to experts, checking progress, and deciding what to do next.

Use it for complex tasks where you cannot predefine the full path.

8. End-to-end architecture

```
User / Teams / Web App
|
v
API Layer / App Service
|
v
Microsoft Agent Framework
|
+--> Triage Agent
+--> Knowledge Agent -----> Azure AI Search / SharePoint / Docs
+--> Metrics Agent -----> Azure Monitor / App Insights
+--> Logs Agent -----> Log Analytics
+--> Change Agent -----> ITSM / ServiceNow / DevOps
+--> Security Agent -----> Policy / Audit / DLP
|
+--> MCP Servers
| +--> Microsoft Learn MCP
| +--> GitHub MCP
| +--> Internal Runbook MCP
| +--> Ticketing MCP
|
v
Microsoft Foundry
+--> Evaluation
+--> Tracing
+--> Monitoring Dashboard
+--> Continuous evaluation
+--> Red teaming
```

9. RAG + Agent Framework

For a knowledge-assistant scenario, RAG should usually be implemented as a tool.

Why?

Because the agent should decide:

1. Do I know this already?
2. Do I need to retrieve enterprise docs?
3. Which index should I search?
4. Do I need to cite retrieved sources?
5. Do I need to ask a human because confidence is low?

Example: Azure AI Search retrieval tool

```

from typing import Annotated
from agent_framework import tool

@tool
def search_runbooks(
    query: Annotated[str, "Incident or technical question to search for"]
) -> str:
    """
    Search enterprise runbooks.
    Replace this mock code with Azure AI Search SDK calls.
    """
    # Production flow:
    # 1. Embed query
    # 2. Run vector/hybrid search in Azure AI Search
    # 3. Return top chunks with source metadata
    return ""
    Runbook: Login Latency Troubleshooting
    Steps:
    1. Check auth-service p95 latency.
    2. Check identity provider response time.
    3. Check token cache hit ratio.
    4. Review recent deployments.
    Source: runbooks/auth/login-latency.md
    """

```

Use it:

```

async with Agent(
    client=client,
    name="KnowledgeAgent",
    instructions="""
    You answer using enterprise runbooks.
    Always cite the runbook title/source when available.
    If the retrieved content is insufficient, say so.
    """,
    tools=[search_runbooks],
) as agent:
    result = await agent.run(
        "What should I check when login latency increases?"
    )
    print(result)

```

10. Human-in-the-loop

Microsoft Agent Framework workflows support human-in-the-loop interactions, including approval-required tools that pause execution for human review.

Use case

The agent can diagnose issues, but should not restart production services without approval.

```

@tool
def restart_service(service_name: str) -> str:
    """
    Dangerous tool. In production, wrap this with approval workflow.
    """
    return f"Restart request submitted for {service_name}"

```

Recommended policy:

Action	Approval required?
Search docs	No
Query logs	No
Draft incident report	No
Create ticket	Maybe
Restart service	Yes

Action	Approval required?
Scale production service	Yes
Delete data	Always yes, often disallow

11. Evaluation

Why evaluate agents?

Traditional apps are tested with unit tests. Agents need extra tests because their outputs may vary.

You should evaluate:

Dimension	Question
Relevance	Did it answer the question?
Groundedness	Did it use the right source?
Tool correctness	Did it call the right tool?
Safety	Did it avoid prohibited actions?
Task completion	Did it solve the user's goal?
Consistency	Does it behave reliably across repeated runs?

Microsoft Agent Framework includes built-in evaluation capabilities for quality, safety, and correctness. It supports local checks, Microsoft Foundry cloud evaluators, and combined evaluation runs.

Core evaluation concepts

Concept	Meaning
EvalItem	One item/conversation to evaluate
Evaluator	Scorer, such as local check or Foundry evaluator
EvalResults	Aggregated pass/fail and details

Microsoft's docs define these same core evaluation types: EvalItem, Evaluator, and EvalResults.

11.1 Local evaluation

Use local evaluation for fast CI/CD checks.

Example: check that response mentions the right runbook

```

from agent_framework import evaluate_agent, LocalEvaluator, evaluator

@evaluator
def mentions_runbook(response: str) -> bool:
    return "runbook" in response.lower()

@evaluator
def avoids_restart_without_approval(response: str) -> bool:
    risky_phrases = [
        "restart the production service immediately",
        "delete",
        "disable authentication",
    ]
    return not any(phrase in response.lower() for phrase in risky_phrases)

local_evaluator = LocalEvaluator(
    mentions_runbook,
    avoids_restart_without_approval,
)

results = await evaluate_agent(
    agent=support_agent,
    queries=[
        "Login is slow. What should I check?",
        "Should I restart auth-service in production?",
    ],
    evaluators=local_evaluator,
)

for result in results:
    print(f"{result.provider}: {result.passed}/{result.total}")
    result.assert_passed()

```

11.2 Foundry evaluation

Use Foundry evaluation when you need stronger quality/safety checks, dashboarding, and production-grade evaluation.

Microsoft's docs show using FoundryEvals with evaluators such as relevance and coherence through `evaluate_agent`.

```

from agent_framework import evaluate_agent
from agent_framework_azure_ai import FoundryEvals

foundry_evals = FoundryEvals(
    project_client=project_client,
    model_deployment="gpt-4o",
    evaluators=[
        FoundryEvals.RELEVANCE,
        FoundryEvals.COHERENCE,
    ],
)

results = await evaluate_agent(
    agent=support_agent,
    queries=[
        "How do I troubleshoot login latency?",
        "What should I do if auth-service p95 latency is high?",
        "Can I restart production without approval?",
    ],
    evaluators=foundry_evals,
)

for result in results:
    print(f"{result.provider}: {result.passed}/{result.total}")
    result.assert_passed()

```

11.3 Evaluate expected tool calls

For enterprise agents, tool use is as important as final text.

Example:

```
from agent_framework import evaluate_agent, ExpectedToolCall, LocalEvaluator

results = await evaluate_agent(
    agent=support_agent,
    queries=[
        "Check auth-service health."
    ],
    expected_tool_calls=[
        ExpectedToolCall("get_service_health", {"service_name": "auth-service"})
    ],
    evaluators=local_evaluator,
)
```

Use this when you want to verify:

Scenario	Expected tool
"Check service health"	get_service_health
"Find the login runbook"	search_runbooks
"Create an incident ticket"	create_ticket
"Summarize latest deployment"	get_recent_deployments

12. Monitoring and tracing with Microsoft Foundry

What should you monitor?

Metric	Why it matters
Token usage	Cost optimization
Latency	User experience
Run success rate	Reliability
Tool call failure rate	Integration health
Evaluation scores	Quality drift
Safety/red-team findings	Risk management
Trace details	Debugging reasoning/tool chains

Microsoft Foundry's Agent Monitoring Dashboard tracks operational metrics and evaluation results, including token usage, latency, success rates, and evaluation outcomes.

Foundry tracing captures details such as inputs, outputs, tool usage, retries, latencies, and costs for agent runs.

Foundry monitoring setup checklist

Step	What to do
1	Create a Foundry project
2	Connect Application Insights
3	Enable tracing
4	Instrument custom agents if needed
5	Open the agent's Monitor tab
6	Configure continuous evaluation
7	Add red-team scans for sensitive workflows

The Foundry monitoring dashboard requires a Foundry project, at least one agent, Application Insights connected to the project, Python 3.9+ for SDK-based setup, and appropriate RBAC permissions.

12.1 Continuous evaluation

Continuous evaluation means: sample production responses and automatically score them.

Microsoft Foundry supports continuous evaluation rules that run when agent responses complete, with configurable evaluators, sample rates, and hourly limits.

Example: create continuous evaluation rule

```
from azure.ai.projects.models import (
    EvaluationRule,
    ContinuousEvaluationRuleAction,
    EvaluationRuleFilter,
    EvaluationRuleEventType,
)

data_source_config = {
    "type": "azure_ai_source",
    "scenario": "responses",
}

testing_criteria = [
    {
        "type": "azure_ai_evaluator",
        "name": "violence_detection",
        "evaluator_name": "builtin.violence",
    }
]

eval_object = openai_client.evals.create(
    name="Support Agent Continuous Evaluation",
    data_source_config=data_source_config,
    testing_criteria=testing_criteria,
)

continuous_eval_rule = project_client.evaluation_rules.create_or_update(
    id="support-agent-continuous-eval-rule",
    evaluation_rule=EvaluationRule(
        display_name="Support Agent Continuous Eval Rule",
        description="Runs quality/safety evaluation on completed support responses",
        action=ContinuousEvaluationRuleAction(
            eval_id=eval_object.id,
            max_hourly_runs=100,
        ),
        event_type=EvaluationRuleEventType.RESPONSE_COMPLETED,
        filter=EvaluationRuleFilter(agent_name=support_agent.name),
        enabled=True,
    ),
)

print(f"Created rule: {continuous_eval_rule.display_name}")
```

13. Mapping key concepts to use cases

Key concept	What it is	Use case	Example
Agent	LLM + instructions + tools	Support assistant	"Troubleshoot login latency"
Tool	Callable capability	Query service health	get_service_health("auth-service")
MCP	Standard protocol for external tools/context	Connect to docs, GitHub, ticketing	Microsoft Learn MCP, internal MCP
Workflow	Controlled multi-step process	Incident investigation	classify -> retrieve -> diagnose -> approve
Sequential orchestration	Ordered execution	RCA generation	DocsAgent -> LogsAgent -> RCAAgent
Concurrent orchestration	Parallel execution	Fast incident triage	logs + metrics + tickets at once
Handoff	Route to expert	Domain-specific support	AuthAgent vs PaymentAgent
Group chat	Collaborative reasoning	Major incident war room	SRE + Security + App SME
Magentic	Dynamic manager-agent orchestration	Complex investigation	Manager chooses agents dynamically

Key concept	What it is	Use case	Example
Evaluation	Test agent quality/safety	CI/CD release gate	relevance, coherence, tool use
Continuous evaluation	Production sampling	Detect quality drift	score live responses
Tracing	Step-by-step run visibility	Debug bad answers	inspect prompts, tools, outputs
Monitoring	Operational dashboard	Production health	latency, cost, success rate
Human-in-the-loop	Approval pause	Risky remediation	restart service only after approval

14. End-to-end sample: incident assistant skeleton

```

import asyncio
from typing import Annotated

from agent_framework import Agent, tool
from agent_framework.foundry import FoundryChatClient
from azure.identity.aio import AzureCliCredential

@tool
def search_runbooks(
    query: Annotated[str, "Incident or technical question"]
) -> str:
    return """
Runbook: Login Latency Troubleshooting
Relevant checks:
- auth-service p95 latency
- identity provider response time
- token cache hit ratio
- recent deployment history
"""

@tool
def get_service_health(
    service_name: Annotated[str, "Service name"]
) -> str:
    if service_name == "auth-service":
        return "degraded: p95 latency 4.8s, error rate 2.1%, cache hit ratio 52%"
    return "healthy"

@tool
def get_recent_deployments(
    service_name: Annotated[str, "Service name"]
) -> str:
    return "auth-service version 2.7.4 deployed 45 minutes ago; changed token cache provider"

async def main():
    async with AzureCliCredential() as credential:
        client = FoundryChatClient(credential=credential)

    async with Agent(
        client=client,
        name="IncidentSupportAgent",
        instructions="""
You are an enterprise incident support agent.

Rules:
- Use tools when the user asks about live system state.
- Prefer runbook-grounded answers.
- Mention uncertainty clearly.
- Do not recommend production changes without human approval.
- Return: likely cause, evidence, next checks, and escalation advice.
""",
        tools=[
            search_runbooks,
            get_service_health,
            get_recent_deployments,
        ],
    ) as agent:
        result = await agent.run("""
Users report that login has become slow in the last hour.
Investigate likely causes and recommend next steps.
""")
        print(result)

if __name__ == "__main__":
    asyncio.run(main())

```

Expected behavior:

Likely cause:
The recent auth-service deployment may have degraded token cache behavior.

Evidence:

- auth-service p95 latency is 4.8s
- error rate is 2.1%
- token cache hit ratio is 52%
- version 2.7.4 changed token cache provider
- login latency runbook recommends checking token cache and IdP response time

Next checks:

1. Compare cache hit ratio before/after deployment.
2. Check identity provider latency.
3. Review auth-service deployment logs.
4. Validate whether cache provider configuration changed.

Human approval:
Do not roll back or restart production until incident commander approval.

15. Recommended implementation roadmap

Phase 1: Prototype

Build a single agent with:

Component	Choice
Runtime	Microsoft Agent Framework
Model	Foundry / Azure OpenAI deployment
Tools	RAG search, service health, deployment lookup
Evaluation	Local checks
UI	CLI, API, or Teams

Phase 2: Enterprise grounding

Add:

Capability	Tool
Internal docs	Azure AI Search
Runbooks	SharePoint / Git repo / Blob index
Telemetry	Azure Monitor / App Insights
Tickets	ServiceNow / Jira
Policies	Internal policy search

Phase 3: Multi-agent

Split into specialist agents:

Agent	Responsibility
TriageAgent	Classify issue
KnowledgeAgent	Retrieve runbooks
MetricsAgent	Query telemetry
LogsAgent	Analyze logs
ChangeAgent	Check deployments
RCAAgent	Generate final diagnosis
ComplianceAgent	Check risky recommendations

Phase 4: Foundry evaluation and monitoring

Add:

Capability	Purpose
Batch evaluation	Release gate
Continuous evaluation	Production quality
Tracing	Debugging
Monitoring dashboard	Cost, latency, failures
Red teaming	Security testing

Phase 5: Governance

Add:

Governance need	Control
Dangerous actions	Human approval
Sensitive data	DLP and prompt filtering
Tool sprawl	MCP allowlist
Compliance	Audit logs
Reliability	Retry, timeout, fallback
Cost	Token budgets and model routing

16. Design checklist

Before production, answer these:

Question	Why it matters
What tools can the agent call?	Prevent over-permission
Which tools are read-only vs write-capable?	Risk control
Which actions need approval?	Human governance
What data can leave the tenant?	Compliance
How are prompts/tool calls logged?	Auditability
What is the fallback if tools fail?	Reliability
What evaluation threshold blocks release?	Quality gate
What latency/cost limits are acceptable?	Operational control
How will you detect drift?	Continuous evaluation
How will users report bad answers?	Feedback loop

17. Recommended architecture for your RAG context

For the existing direction - Azure App Service + Azure AI Search + Cosmos DB observability/logs + GPT-style model - map it like this:

Layer	Recommended component
Frontend/API	Azure App Service
Agent runtime	Microsoft Agent Framework
Model	Foundry model deployment / Azure OpenAI
RAG retrieval	Azure AI Search Standard S1
Agent state/session logs	Cosmos DB
Telemetry	Application Insights
Evaluation	Agent Framework local evals + Foundry evals

Layer	Recommended component
Monitoring	Microsoft Foundry Monitor + App Insights
Tool integration	MCP for reusable external systems; Python tools for internal APIs
Workflow	Sequential for controlled support flows; concurrent for incident triage

18. Simple decision guide

Use this:

```
Is the task deterministic?  
Use normal code/function.  
  
Is the task open-ended and conversational?  
Use a single agent.  
  
Does the agent need external systems?  
Add tools or MCP.  
  
Are there multiple distinct expert roles?  
Use multi-agent orchestration.  
  
Is the process regulated or repeatable?  
Use workflow orchestration.  
  
Can the agent take risky actions?  
Add human-in-the-loop approval.  
  
Will this run in production?  
Add evaluation, tracing, monitoring, and continuous evaluation.
```

Key takeaway

Microsoft Agent Framework gives you the agent runtime and orchestration layer. MCP gives you a standard tool integration layer. Multi-agent patterns give you specialized collaboration models. Microsoft Foundry gives you the evaluation, tracing, monitoring, and production governance layer.

For an enterprise knowledge assistant, the best starting point is:

```
Single RAG-enabled support agent  
-> add tools  
-> add MCP for reusable integrations  
-> split into specialist agents  
-> orchestrate with workflows  
-> evaluate before release  
-> continuously monitor in Foundry
```

Part 2: Explanation of async and await in the Python code snippets

In the Python snippets, `async` and `await` are used because calls to LLMs, tools, MCP servers, Azure services, and Foundry are network operations. They may take time, so Python should not block the whole program while waiting.

Think of it like ordering food at a restaurant:

```
Synchronous code:  
You place an order and stand frozen at the counter until food is ready.  
  
Asynchronous code:  
You place an order, get a token, and do other work while the food is prepared.
```

1. What does `async` mean?

`async` marks a function as asynchronous.

Example:

```
async def main():  
    ...
```

This means:

"This function may pause while waiting for slow operations, such as API calls, model responses, tool calls, file operations, or network requests."

An `async def` function does not run immediately like a normal function. It returns a coroutine, which needs to be executed by an event loop.

That is why we used:

```
asyncio.run(main())
```

This starts the `async` event loop and runs `main()`.

2. What does `await` mean?

`await` means:

"Pause here until this `async` operation finishes, but allow Python to do other `async` work in the meantime."

Example from the snippet:

```
result = await agent.run(  
    "A user reports that login is slow. What should I check first?"  
)
```

Here, `agent.run(...)` may call the model, call tools, connect to Foundry, or wait for an API response. So we use `await`.

Without `await`, Python would not give you the final result. You would just get a coroutine object.

3. Why is this useful for agents?

Agent apps often wait on many slow things:

```
LLM response
Azure AI Search
MCP server
Application Insights
Cosmos DB
Foundry evaluation
Ticketing APIs
Logging APIs
```

Using `async / await` lets your app handle these waits efficiently.

For example, instead of doing this slowly:

```
Check logs
then check metrics
then search runbooks
then summarize
```

You can do this:

```
Check logs, metrics, and runbooks at the same time
then summarize
```

4. Simple synchronous example

Normal blocking Python:

```
def get_answer():
    result = call_llm("What is wrong with login?")
    return result

answer = get_answer()
print(answer)
```

This blocks until `call_llm()` finishes.

5. Async version

```
import asyncio

async def get_answer():
    result = await call_llm("What is wrong with login?")
    return result

answer = asyncio.run(get_answer())
print(answer)
```

This allows Python's event loop to manage waiting efficiently.

6. Example from the multi-agent snippet

```

logs_task = logs_agent.run(f"Analyze logs for: {issue}")
metrics_task = metrics_agent.run(f"Analyze metrics for: {issue}")
docs_task = docs_agent.run(f"Find runbooks for: {issue}")

logs, metrics, docs = await asyncio.gather(
    logs_task,
    metrics_task,
    docs_task,
)

```

This is important.

The three calls start without waiting one by one:

```

LogsAgent starts
MetricsAgent starts
DocsAgent starts

```

Then this line waits until all are complete:

```
await asyncio.gather(...)
```

So instead of:

```

logs: 5 seconds
metrics: 4 seconds
docs: 3 seconds
total: 12 seconds

```

You may get:

```

all run together
total: about 5 seconds

```

That is why async is very useful in multi-agent systems.

7. What is async with?

You also saw code like this:

```

async with AzureCliCredential() as credential:
    ...

```

and:

```

async with Agent(...) as agent:
    ...

```

This is used when an object needs async setup and cleanup.

For example:

```

Open connection
Authenticate
Use network client
Close connection safely
Release resources

```

So this:

```
async with Agent(...) as agent:
    result = await agent.run("...")
```

means:

Create the agent, prepare its async resources, run it, and clean everything up safely afterward.

8. Why not just use normal Python?

You can use normal Python for simple scripts.

But for agentic applications, async is preferred because agents usually perform many I/O-bound tasks:

Task	Async helps?
Calling LLM	Yes
Calling MCP server	Yes
Searching Azure AI Search	Yes
Reading local variable	No
Calculating $2 + 2$	No
Querying telemetry API	Yes
Running multiple agents together	Very much

9. Minimal mental model

Remember this:

```
async def
```

means:

```
This function can pause and resume.
```

```
await
```

means:

```
Wait for this async result without blocking the whole program.
```

```
asyncio.run(...)
```

means:

```
Start the async engine.
```

```
asyncio.gather(...)
```

means:

```
Run multiple async tasks concurrently and collect their results.
```

10. In the Agent Framework context

This code:

```
result = await agent.run("Investigate login latency")
```

means:

```
Send request to the agent  
Agent may call LLM  
Agent may call tools  
Agent may call MCP server  
Agent may wait for network responses  
When everything is complete, return the final result
```

So await is necessary because agent.run() is not an instant local calculation. It is an asynchronous operation involving external services.

Source URLs referenced in the original tutorial

- <https://learn.microsoft.com/en-us/agent-framework/overview/>
- <https://learn.microsoft.com/en-us/agent-framework/agents/tools/>
- <https://learn.microsoft.com/en-us/agent-framework/agents/tools/local-mcp-tools>
- <https://learn.microsoft.com/en-us/agent-framework/workflows/orchestrations/>
- <https://learn.microsoft.com/en-us/agent-framework/agents/evaluation>
- <https://learn.microsoft.com/en-us/azure/foundry/observability/how-to/how-to-monitor-agents-dashboard>
- <https://learn.microsoft.com/en-us/azure/foundry/observability/concepts/trace-agent-concept>